

Database development with PL/SQL

Reference reading



- [Auditing Enhancements \(Audit Policies and Unified Audit Trail\) in Oracle Database 12c Release 1 \(12.1\)](#)

Lecture 10 - Advanced Auditing in PLSQL

Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | eric.maniraguha@auca.ac.rw | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

April 2025

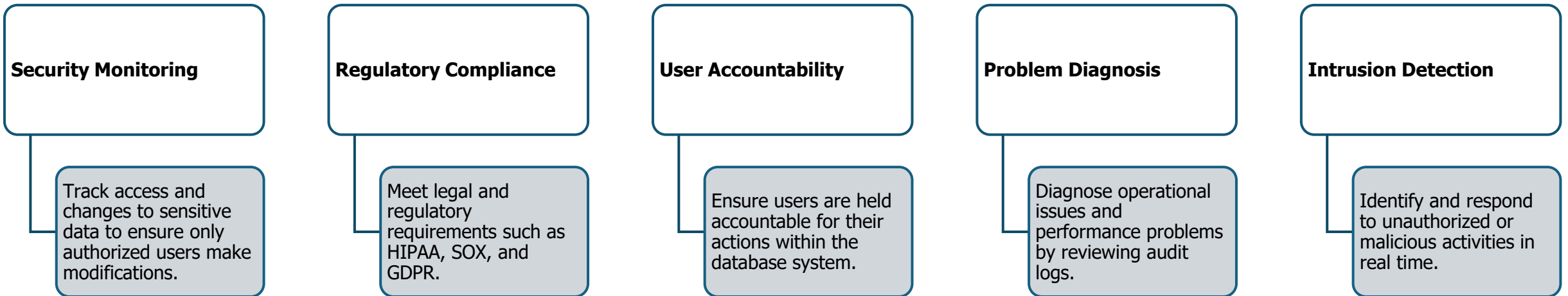
The objectives of Auditing in Databases | Key Components of Database Auditing

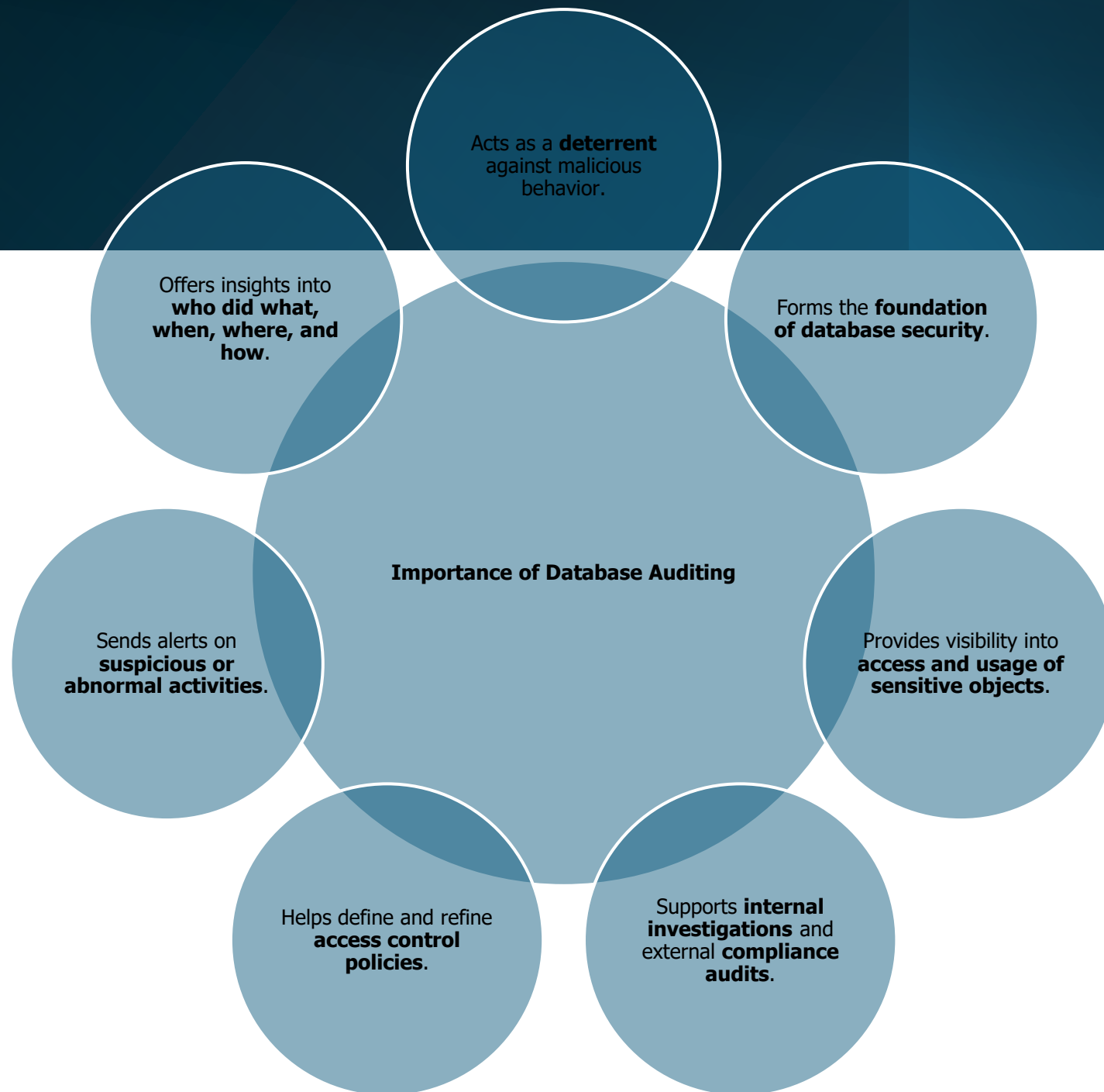
Auditing in databases refers to monitoring, recording, and analyzing user activities within the database. This includes tracking SQL operations, user access, and other actions that involve data. It serves to ensure accountability, compliance, and the detection of unauthorized or suspicious activities.

Activities Monitored:	Actions on specific schemas, tables, or rows.
	Successful or failed database access attempts.
	Privileged user actions (e.g., SYSDBA operations).
Configuration:	Enabling auditing through initialization parameters (e.g., AUDIT_TRAIL).
	Setting up audit policies to target specific activities or data.
Audit Record Locations:	Database audit trail (e.g., SYS.AUD\$).
	Operating system files (e.g., UNIX syslog or Windows Event Viewer).
Auditing Types:	Standard Auditing: Focuses on SQL statements, privileges, and schema objects.
	Fine-Grained Auditing: Offers detailed auditing at the data level (e.g., actions based on specific conditions like IP address).
Types of Users in Auditing:	Database Users: Traditional users who directly interact with the Oracle database.
	Nondatabase Users: Users of applications that interact with the database. These users are identified in the database using the CLIENT_IDENTIFIER attribute.
	Auditing nondatabase users is more complex as they interact indirectly through the application.

Objectives of Auditing in Databases

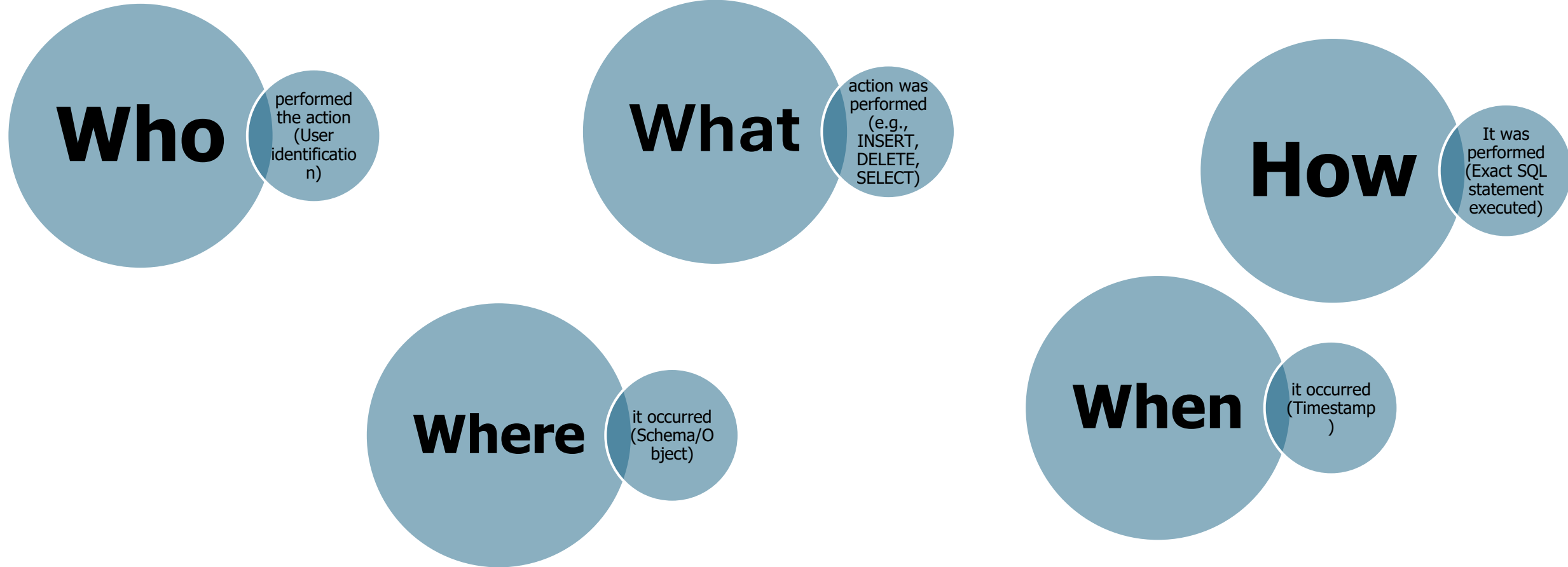
Auditing in databases refers to monitoring, recording, and analyzing user activities within the database. This includes tracking SQL operations, user access, and other actions that involve data. It serves to ensure accountability, compliance, and the detection of unauthorized or suspicious activities.





Audit Trail: Key Information Captured

A well-maintained audit trail includes:



Who Can Perform Auditing?

The ability to configure and manage database auditing is typically reserved for:

Database Administrators (DBAs):

- They have full access to define audit policies, monitor audit logs, and respond to incidents.

Security Officers or Compliance Teams:

- May have read-only access to audit logs for review, auditing, or investigation purposes.

Application Auditors (in some environments):

- Developers or auditors with special permissions to monitor specific tables or actions for business logic compliance.

Note:

- Proper role-based access control (RBAC) should be enforced to ensure that audit data itself is protected from unauthorized access or tampering.

Key Components of Database Auditing

Audit Trail: The log where audit records are stored.

Audit Policies: Define what actions should be audited.

Audited Actions: Actions like SELECT, INSERT, DELETE, UPDATE, logins, etc.

Audit Options: Settings that specify conditions like BY USER or BY OBJECT.

Audit Types:

Standard Auditing
(deprecated in newer versions)

Unified Auditing
(introduced in Oracle 12c)

Component	Description
Activities Monitored	Object access, logins, SYSDBA actions
Configuration	Audit parameters, policy creation and enablement
Audit Record Locations	SYS.AUD\$, UNIFIED_AUDIT_TRAIL, OS logs
Auditing Types	Standard, FGA, Unified, Vault-specific auditing
User Types	Direct DB users, indirect (app) users tracked via CLIENT_IDENTIFIER

Oracle Audit Vault

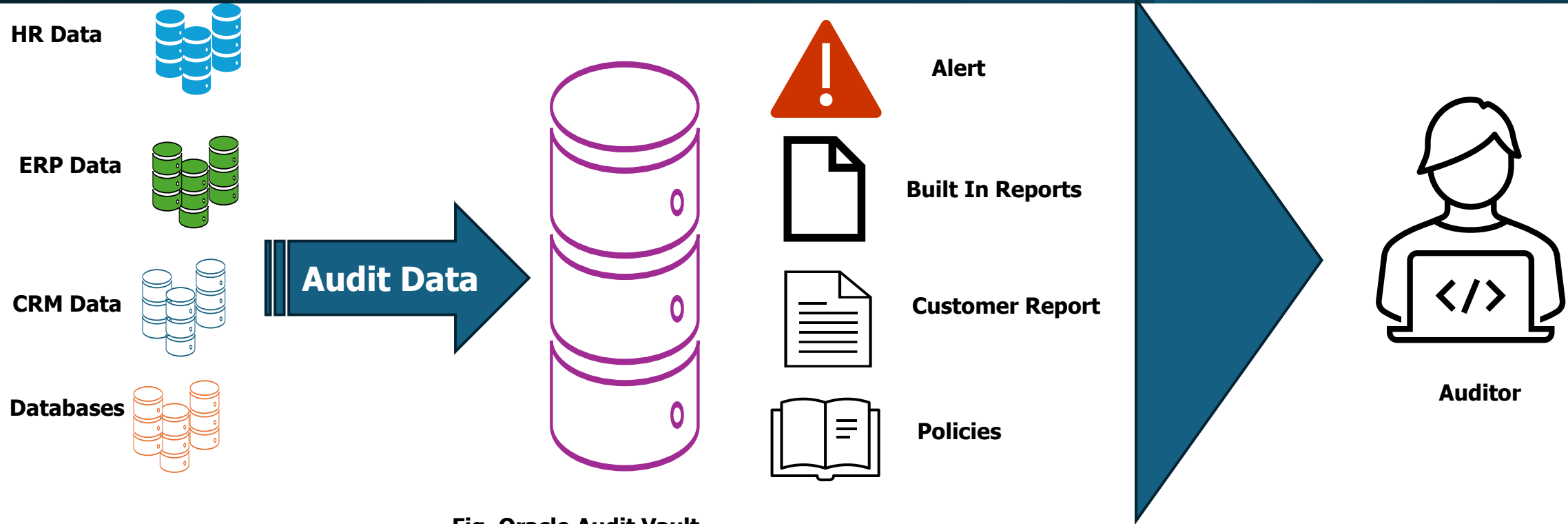


Fig. Oracle Audit Vault

Purpose:

This diagram highlights how Oracle Audit Vault centralizes auditing for various data systems, enabling streamlined monitoring, compliance enforcement, and auditing for organizations. It shows the end-to-end process of collecting, storing, analyzing, and reporting data to assist auditors in decision-making.

Oracle Audit Vault – Diagram Key Elements and Workflow

1. Data Sources (Left side of the diagram) - The monitored systems include:

HR Data: Employee-related databases.

ERP Data: Enterprise resource planning systems.

CRM Data: Customer relationship management systems.

Databases: General-purpose databases storing organizational data.

These systems generate **audit data**, which includes logs of user activity, data access, and any changes made.

2. Audit Data Collection and Storage (Middle section):

Audit Data: The logs and audit trails from the data sources are collected and transported to a central **Audit Data Repository** (the pink database icon).

This repository serves as the backbone for securely storing all audit logs in a structured format for further analysis.

3. Processing and Insights (Middle elements on the right of the repository): -

The stored audit data is processed to generate actionable insights:

Alerts: Notifications triggered by suspicious activities, such as unauthorized access or policy violations.

Built-In Reports: Predefined, standard reports for common auditing tasks (e.g., user activity logs, failed login attempts).

Customer Reports: Customized reports tailored to specific compliance or business requirements.

Policies: Rules and monitoring criteria configured to enforce security and regulatory compliance.

4. Output for Auditors (Right side of the diagram):

The processed insights (alerts, reports, and policies) are made available to the **Auditors**.

Auditors use this information to:

- ✓ Monitor system activities.
- ✓ Investigate any violations or unusual behavior.
- ✓ Ensure compliance with security standards and regulatory requirements.

Purpose of the System:

Oracle Audit Vault centralizes auditing and monitoring across multiple systems, providing organizations with:

- A secure and consolidated way to manage audit logs.
- The ability to detect suspicious activities in real time.
- Tools for creating reports to comply with internal policies and external regulations.

This end-to-end process ensures effective oversight of organizational data, reducing security risks and improving compliance.

Database Auditing Process: Step-by-Step Guide

Set the foundation for a successful audit.

- **Define Objectives**
What is the goal? (e.g., ensure compliance, detect fraud, assess system efficiency)
- **Understand the Scope**
Which systems, processes, or users will be audited?
- **Gather Information**
Review policies, system documentation, user roles, and regulatory requirements.
- **Develop an Audit Plan**
Timeline, resources, tools, and audit team responsibilities.

1. Planning the Audit

2. Preparation

Organize your team and get ready.

- **Notify Stakeholders**
Inform affected users and departments about the upcoming audit.
- **Identify Key Risks**
Focus on areas prone to misuse or non-compliance.
- **Assign Roles**
Clarify who does what within the audit team.

Carry out the audit and gather evidence.

- **Data Collection**
Use logs, system reports, interviews, or observations.
- **Testing and Verification**
Assess if controls and procedures are working as intended.
- **Analysis**
Compare findings with standards, policies, or industry benchmarks.

3. Execution

4. Evaluation

Assess the collected data.

- **Summarize Findings**
Clearly present what was discovered.
- **Identify Gaps**
Point out violations, risks, or inefficiencies.
- **Determine Impact**
Estimate how each issue affects the organization (e.g., security risk, financial loss, legal exposure).

Share results and make recommendations.

- **Draft the Report**
Include clear findings, conclusions, and next steps.
- **Discuss with Stakeholders**
Get feedback and clarify any misunderstandings.
- **Finalize Report**
Finalize after incorporating feedback from stakeholders.

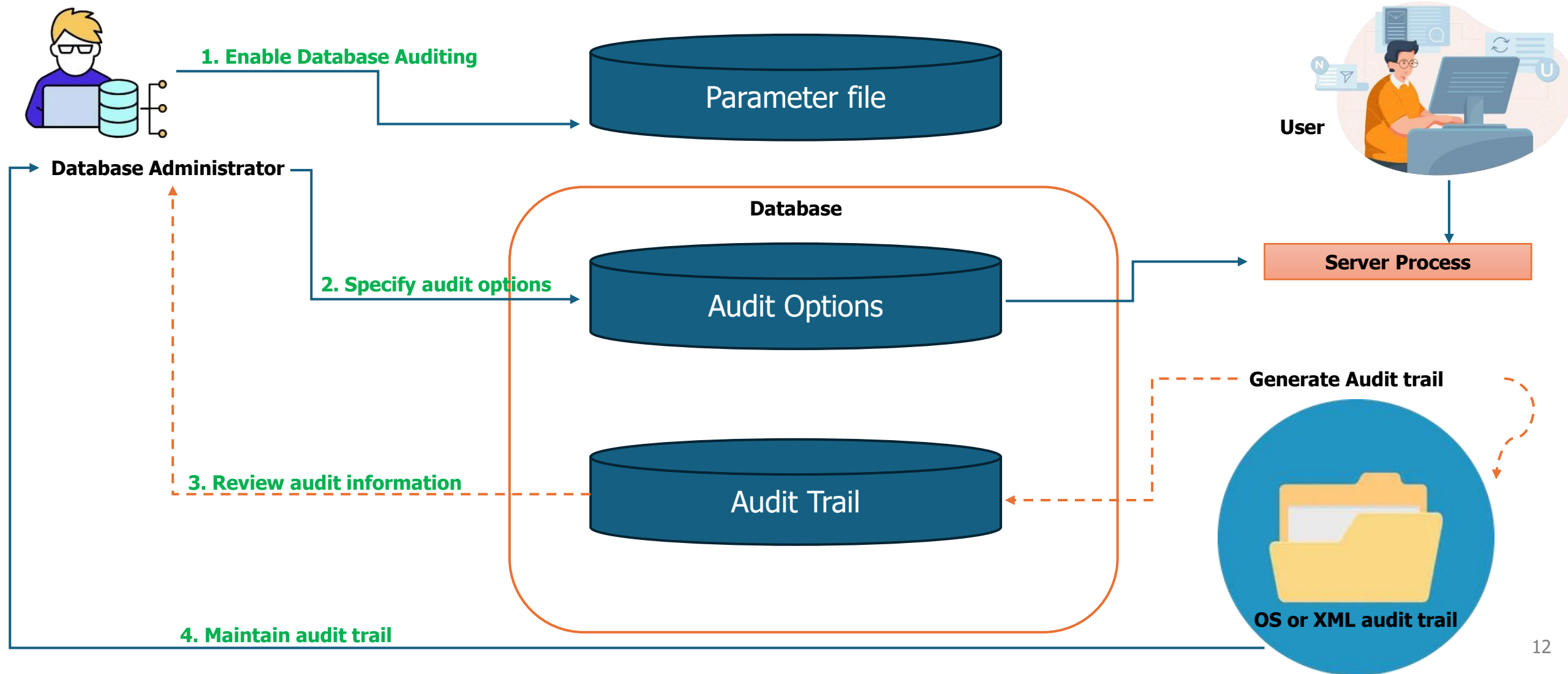
5. Reporting

6. Follow-Up

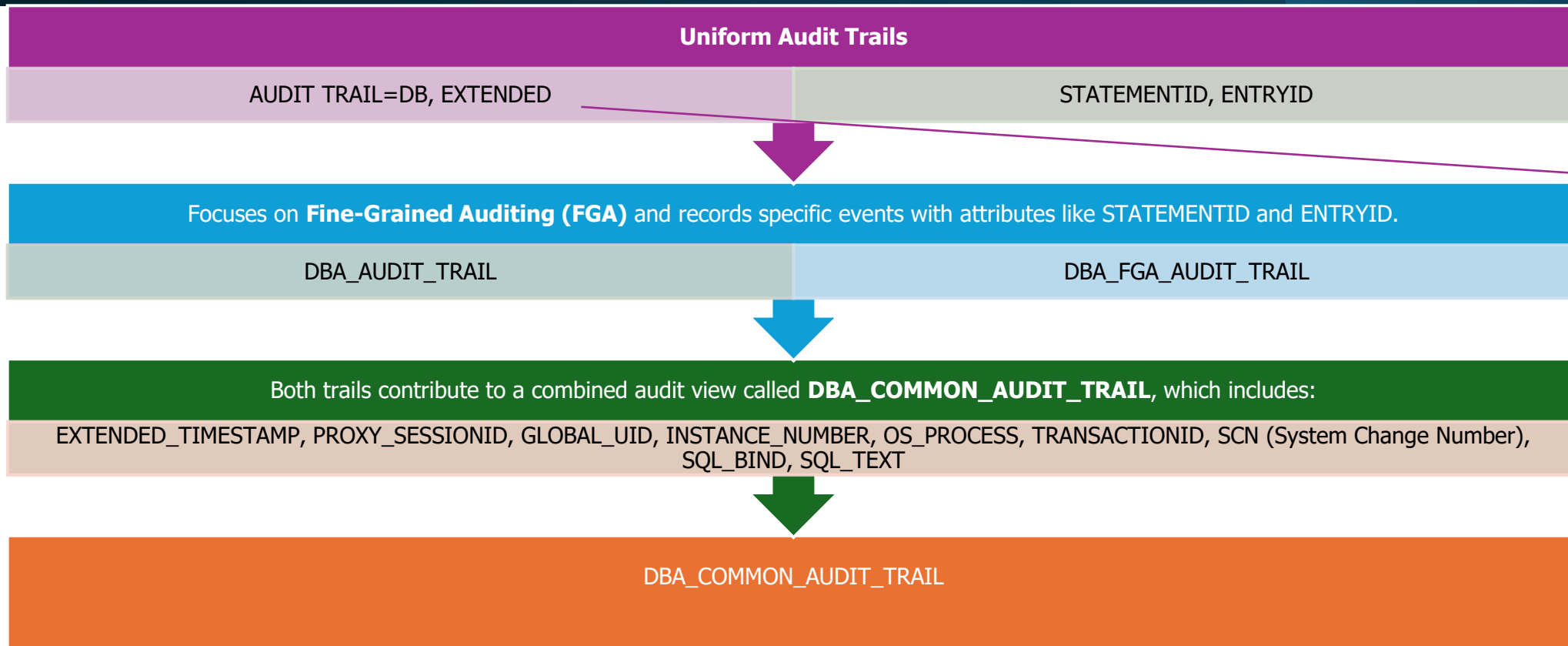
Ensure changes are made and lessons are learned.

- **Monitor Implementation**
Verify that corrective actions have been applied.
- **Reassess**
If needed, conduct another audit or mini-review.
- **Continuous Improvement**
Use what you've learned to improve future audit processes and security practices.

Standard Database Auditing



Uniform Audit Trails



Monitoring:

- Use **DBA_AUDIT_TRAIL** and **DBA_FGA_AUDIT_TRAIL** to review detailed logs.
- Consolidate and analyze using **DBA_COMMON_AUDIT_TRAIL** for a unified view.

This setup allows for comprehensive monitoring, ensuring accountability and enabling root-cause analysis for database events.

Tracks detailed database activity and stores it in two audit trails

Uniform Audit Trails Framework

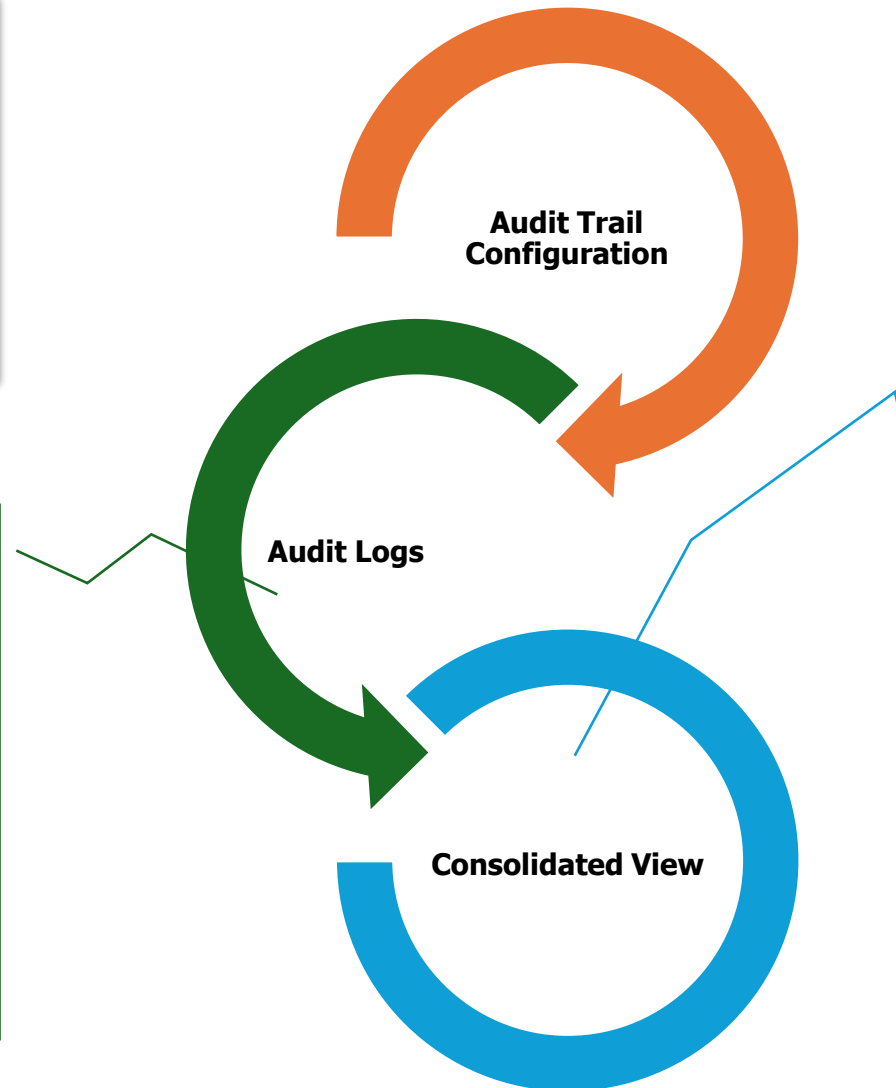


1. Audit Trail Configuration:

- Set **AUDIT_TRAIL=DB, EXTENDED** to enable:
 - Comprehensive auditing of SQL statements and database events.
 - Detailed logging of extended information.

2. Audit Logs:

- **DBA_AUDIT_TRAIL:**
 - Captures general database audit events (e.g., user actions, logins).
- **DBA_FGA_AUDIT_TRAIL:**
 - Logs **Fine-Grained Auditing (FGA)** activities, including:
 - ✓ **StatementID:** A unique identifier for each SQL statement.
 - ✓ **EntryID:** A sequential record number for precise tracking.



3. Consolidated View

DBA_COMMON_AUDIT_TRAIL combines data from both audit trails and includes extended information:

- **EXTENDED_TIMESTAMP:** Accurate time of the event.
- **PROXY_SESSIONID:** Tracks sessions with proxy users.
- **GLOBAL_UID:** Unique global user identifier.
- **INSTANCE_NUMBER:** Database instance where the event occurred.
- **OS_PROCESS:** Operating system process ID for the session.
- **TRANSACTIONID:** Identifies related transactions.
- **SCN (System Change Number):** Database state identifier.
- **SQL_BIND:** Details of bind variables in the statement.
- **SQL_TEXT:** Full SQL query text executed.

Default Auditing

Privilege	Audit Type
ALTER ANY PROCEDURE	SYSTEM, BY ACCESS
ALTER ANY TABLE	ROLE, BY ACCESS
ALTER DATABASE	SYSTEM
ALTER PROFILE	SYSTEM
ALTER SYSTEM	SYSTEM
ALTER USER	SYSTEM
AUDIT SYSTEM	SYSTEM
CREATE ANY JOB	SYSTEM
CREATE ANY LIBRARY	SYSTEM
CREATE ANY PROCEDURE	SYSTEM
CREATE ANY TABLE	SYSTEM
CREATE EXTERNAL JOB	SYSTEM
CREATE PUBLIC DATABASE LINK	SYSTEM
CREATE SESSION	SYSTEM
CREATE USER	SYSTEM
GRANT ANY OBJECT PRIVILEGE	SYSTEM
GRANT ANY PRIVILEGE	SYSTEM
GRANT ANY ROLE	SYSTEM
DROP ANY PROCEDURE	SYSTEM
DROP ANY TABLE	SYSTEM
DROP PROFILE	SYSTEM
DROP USER	SYSTEM
EXEMPT ACCESS POLICY	SYSTEM

Privileges Audited by Default

Insight: Default auditing ensures that sensitive operations, such as granting roles, altering system configurations, or creating/dropping tables and procedures, are monitored. This is crucial for identifying unauthorized or unintended changes.

	Statement	Audit Type
Statements Audited by Default	SYSTEM AUDIT	BY ACCESS
	ROLE	BY ACCESS

Specifying Audit Options in Oracle Audit Vault

Oracle provides the AUDIT and NOAUDIT SQL statements to enable or disable auditing of various activities. These options help track user actions for security and compliance purposes. Below is a categorized list of auditing commands:

1. SQL Statement Auditing

Audit Type	Command	Description	Disable Command
TABLE statements	AUDIT TABLE;	Audits all TABLE statements executed in the database.	NOAUDIT TABLE;
LOCK statements	AUDIT LOCK;	Audits all LOCK statements executed in the database.	NOAUDIT LOCK;

2. System Privileges Auditing

Audit Type	Command	Description	Disable Command
Privilege usage auditing	AUDIT SELECT ANY TABLE, CREATE ANY TRIGGER;	Audits sessions using SELECT ANY TABLE or CREATE ANY TRIGGER privileges.	NOAUDIT SELECT ANY TABLE, CREATE ANY TRIGGER;
User-specific auditing	AUDIT SELECT ANY TABLE BY HR BY SESSION;	Audits SELECT ANY TABLE for the HR user, grouped by session.	NOAUDIT SELECT ANY TABLE BY HR BY SESSION;

3. Object Auditing

Audit Type	Command	Description	Disable Command
All operations on a table	AUDIT ALL ON HR.EMPLOYEES;	Audits all operations on the HR.EMPLOYEES table.	NOAUDIT ALL ON HR.EMPLOYEES;
Specific operations	AUDIT UPDATE, DELETE ON HR.EMPLOYEE BY ACCESS;	Audits UPDATE and DELETE operations on HR.EMPLOYEE, recorded per access.	NOAUDIT UPDATE, DELETE ON HR.EMPLOYEE BY ACCESS;
Rename operation	AUDIT RENAME ON HR.EMPLOYEE;	Audits all RENAME operations on the HR.EMPLOYEE table.	NOAUDIT RENAME ON HR.EMPLOYEE;

Fine Grained Auditing (FGA)

Fine-Grained Auditing (FGA) allows you to audit specific actions at a detailed level by applying policies to table objects. It is used to monitor and log access based on conditions or specific rows in a table.

Key Features of FGA:

Granularity: Audits specific rows or conditions in a table, not just general access.

Policies: Auditing is driven by policies that define the conditions under which auditing should occur.

Custom Conditions: You can specify predicates (conditions) that determine when an audit is triggered.

Custom Handlers: Optionally, an audit event handler can execute PL/SQL code when an audit condition is met.

Policy: AUDIT_EMPS_SALARY

SELECT name, salary FROM employees WHERE department_id = 10;



Employees

Fine Grained Auditing (FGA)

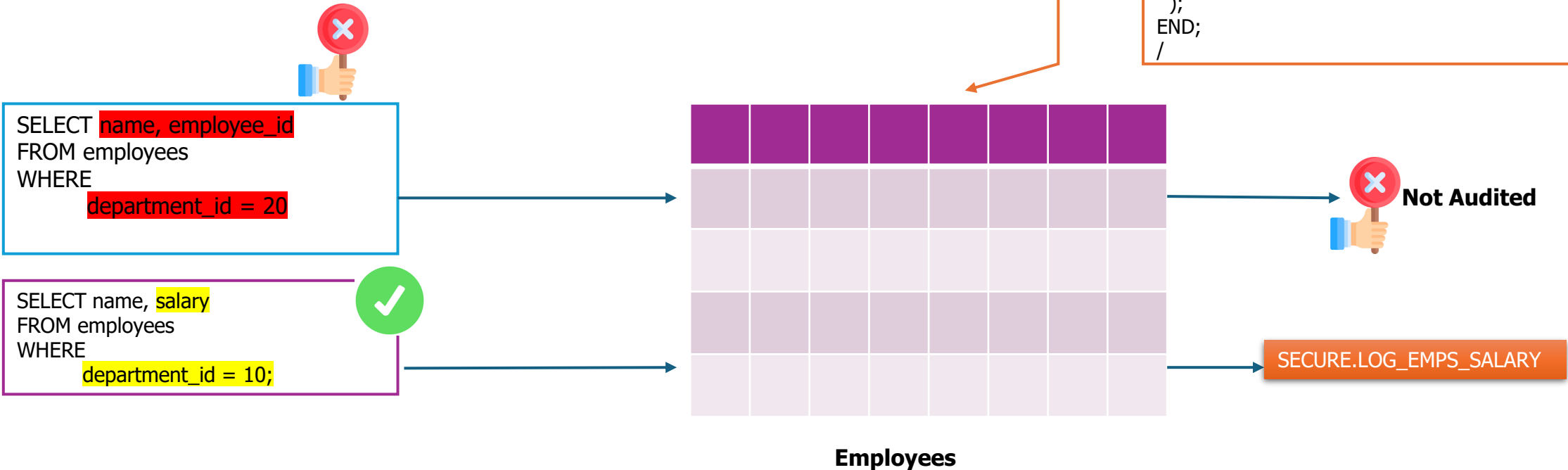
What is Fine-Grained Auditing (FGA)?

FGA allows **precise control over what gets audited** in the database by applying conditions and focusing on specific columns.

It is especially useful when monitoring **sensitive data access** without logging every query.

Component	Description
Audit Criteria	The condition or rule that must be met for auditing to occur (e.g., salary > 10000).
Audit Action	The database operation(s) to audit: <i>SELECT, INSERT, UPDATE, DELETE.</i>
Created Using	<i>DBMS_FGA.ADD_POLICY</i> procedure in Oracle.

```
BEGIN
  DBMS_FGA.ADD_POLICY(
    object_schema => 'HR',
    object_name   => 'EMPLOYEES',
    policy_name   => 'SALARY_AUDIT',
    audit_condition => DEPARTMENT_ID = 10,
    audit_column  => 'SALARY, COMMISSION_PCT',
    handler_schema => 'SECURE',
    handler_module => 'log_emps_salary',
    enable        => TRUE,
    statement_types => 'SELECT, UPDATE'
  );
END;
/
```



Audited DML Statement: Consideration

Fine-Grained Auditing (FGA) Overview

Definition: FGA audits database operations based on specific conditions and referenced columns, providing targeted monitoring for sensitive data.

Audited Actions

Operations like SELECT, INSERT, and UPDATE are audited if conditions are met or specific columns are referenced.

DELETE statements are **always audited**, regardless of conditions or columns.

MERGE Statement Auditing

Behavior: The MERGE statement triggers audits for its underlying INSERT, UPDATE, and DELETE operations as independent actions.

Implications:

Each operation within a MERGE is checked against FGA policies.

Logs must capture operation type (e.g., INSERT or UPDATE) and relevant details like affected rows or columns.

Key Considerations

Policy Setup: Ensure FGA policies cover all relevant operations (INSERT, UPDATE, DELETE) triggered by MERGE.

Performance: Target sensitive columns and conditions to minimize overhead.

Audit Clarity: Maintain detailed logs distinguishing the operation type and the affected data.

This ensures robust, compliant, and efficient auditing of both direct and composite operations like MERGE.



```
UPDATE hr.employees SET salary = 1000 WHERE commission_pct = .2;
```

```
UPDATE hr.employees SET salary = 1000 WHERE employee_id = 200;
```



Not audited because none of the records involved are for department 10

Oracle Fine-Grained Auditing (FGA) Guidelines

Audit Conditions and Columns:

- Audit All Rows:** Use a **null audit condition** (no condition specified).
- Audit All Columns:** Use a **null audit column** to apply the policy to all columns in the table.

Policy Creation:

- Unique Policy Names:** Each FGA policy must have a unique name within the schema.
- Pre-existing Tables/Views:** The table or view to be audited must exist before creating the policy.

Error Handling:

- Invalid Syntax:** If the audit condition has invalid syntax, accessing the audited object will raise an ORA-28112 error.
- Nonexistent Columns:** No rows are audited if the specified column in the policy does not exist in the table.
- Missing Event Handlers:** If the specified event handler (e.g., a PL/SQL procedure) does not exist, no error occurs, and the audit record is still generated.

Best Practices:

- Ensure audit conditions and columns are carefully tested to avoid unintended errors or gaps in auditing.
- Regularly validate the existence and functionality of event handlers tied to FGA policies.

By following these guidelines, you can implement robust and error-free auditing policies in Oracle Database.

Maintaining the Audit Trail in Oracle 12c Database (FGA/Standard Auditing)

Best Practices for Maintaining the Audit Trail in Oracle 12c Database (FGA/Standard Auditing):

To ensure the integrity and reliability of your audit trail in Oracle Database, follow these best-practice guidelines.

Review and Archive Old Records:

Regularly review audit records to identify any suspicious activities or patterns.

Archive historical audit data to a secure location for long-term storage, ensuring compliance with regulatory requirements.

Prevent Storage Problems:

Monitor the available space in the tablespaces used for audit data to avoid storage issues.

Implement partitioning or purging mechanisms to manage the volume of audit logs effectively.

Use Oracle's DBMS_AUDIT_MGMT package to automate audit trail cleanup and management tasks.

Avoid Loss of Records:

Enable appropriate logging mechanisms to capture all relevant audit activities, such as Fine-Grained Auditing (FGA) for detailed monitoring.

Regularly back up the audit trail to prevent data loss due to hardware failures or unexpected events.

Configure alerts for critical thresholds to proactively address storage or system issues that could impact audit logging.

By adhering to these practices, you can maintain a reliable and secure audit trail that supports both security and compliance needs within your Oracle Database environment.

1. Configuring the Audit Trail (1/2)

```
C:\Users\Eric>sqlplus sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA
```

```
SQL*Plus: Release 21.0.0.0.0 - Production on Sun Nov 17 14:08:00 2024  
Version 21.3.0.0.0
```

```
Copyright (c) 1982, 2021, Oracle. All rights reserved.
```

```
Connected to:  
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production  
Version 21.3.0.0.0
```

```
SQL>  
SQL>  
SQL>  
SQL> SHOW PDBS;
```

CON_ID	CON_NAME	OPEN MODE	RESTRICTED
2	PDB\$SEED	READ ONLY	NO
3	ORACLE21DBCPDB	MOUNTED	
5	PLSQL2024	READ WRITE	NO

```
SQL>  
SQL>  
SQL> show con_name;
```

```
CON_NAME
```

```
-----  
CDB$ROOT
```

-- Connect to the database

SQL> sqlplus sys as SYSDBA or

C:\Users\Eric> sqlplus sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA

-- Check all pluggable database

SQL> SHOW PDBS;

-- Check all pluggable database

SQL> SHOW CON_NAME;

2. Understand Where Audit Data is Stored

AUD\$ table: The audit records are stored in the **SYS.AUD\$** table within the database.

DBA_AUDIT_TRAIL view: Oracle provides the **DBA_AUDIT_TRAIL** view, which you can query to access audit records in a user-friendly format.

Operating System or XML files: If you set up auditing to log data externally (e.g., **AUDIT_TRAIL = OS** or **AUDIT_TRAIL = XML**), audit files will be generated in the specified directory.

```
/*  
2. Query Audit Data  
Use the following SQL queries to interact with the audit trail:  
Check All Audit Records:  
*/  
SELECT * FROM DBA_AUDIT_TRAIL;
```

```
/*  
This will show all audit records logged in the database.  
Filter Audit Records by User:  
*/  
SELECT * FROM DBA_AUDIT_TRAIL WHERE USERNAME = 'USERNAME';
```

```
/*  
Filter by Action (e.g., Login)  
*/  
SELECT * FROM DBA_AUDIT_TRAIL WHERE ACTION_NAME = 'LOGON';
```

```
/*  
Check Audit Policies:  
*/  
SELECT * FROM DBA_OBJ_AUDIT_OPTS;
```

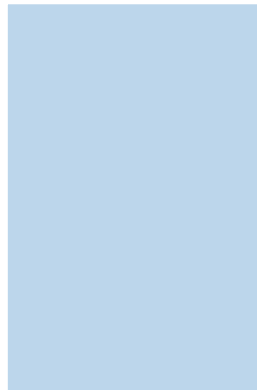
3. Monitor Audit log



For real-time monitoring of audit logs, consider using Oracle tools such as:



Oracle Enterprise Manager (OEM): Provides a graphical interface for monitoring and managing audit logs.



PL/SQL scripts or shell scripts: Create automated scripts to extract and analyze audit data periodically.

4. Manage Audit Logs

For real-time monitoring of audit logs, consider using Oracle tools such as:

```
/*  
Archive Audit Records: Export the audit logs periodically:  
*/  
  
expdp system/your_password schemas=SYS tables=AUD$ directory=DATA_PUMP_DIR  
dumpfile=audit_logs.dmp logfile=audit_logs.log;
```

```
/*  
Purge Audit Records: Use the DBMS_AUDIT_MGMT package to clean up old audit data:  
*/  
  
BEGIN  
  DBMS_AUDIT_MGMT.INIT_CLEANUP(  
    AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_AUD_STD,  
    USE_LAST_ARCH_TIMESTAMP => TRUE  
  );  
  DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(  
    AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_AUD_STD,  
    USE_LAST_ARCH_TIMESTAMP => TRUE  
  );  
END;  
/
```

Configure Specific Audit Policies & Use Unified Auditing (Optional)

5. To audit specific actions or objects

```
/*  
Audit SQL Statements:  
*/  
AUDIT SELECT TABLE BY USERNAME;  
  
/*  
Audit Object Access:  
*/  
AUDIT INSERT, UPDATE, DELETE ON SCHEMA.TABLE;
```

6. Use Unified Auditing (Optional)

```
/*  
If your database is configured for Unified Auditing, use UNIFIED_AUDIT_TRAIL  
*/  
SELECT * FROM UNIFIED_AUDIT_TRAIL WHERE EVENT_TIMESTAMP > SYSDATE - 1;  
  
/*  
To enable specific audit policies:  
*/  
CREATE AUDIT POLICY login_audit_policy  
  ACTIONS LOGON, LOGOFF;  
AUDIT POLICY login_audit_policy;
```

Access Audit Vault (Optional) & Security Best Practices

7. Access Audit Vault (Optional): If you have configured Oracle Audit Vault, use the Audit Vault interface to:

View, manage, and report on audit logs.

Set up alerts for suspicious activities.

8. Security Best Practices

Regularly review audit logs for suspicious activities.

Limit access to audit data to authorized users only.

Rotate and archive logs to comply with regulatory requirements.

By leveraging these approaches, you can effectively interact with and manage your audit trail.

End and Conclusion

Oracle Audit Vault plays a critical role in strengthening database security by providing centralized auditing and monitoring of user activity across the enterprise. Through **SQL Statement Auditing**, **System Privileges Auditing**, and **Object Auditing**, organizations can effectively track access, detect anomalies, and ensure compliance with internal policies and external regulations.

By leveraging audit options such as AUDIT and NOAUDIT, database administrators can:

- Monitor sensitive operations and user activities.
- Detect unauthorized access or misuse of system privileges.
- Maintain a comprehensive trail for accountability and investigation.
- Enhance visibility into data usage across the system.
- Facilitate compliance with standards such as GDPR, HIPAA, and SOX.

In conclusion, **Oracle Audit Vault and Database Firewall** provide a proactive approach to database security and governance. When implemented effectively, they help protect critical assets, support regulatory compliance, and promote trust in data management systems.

Security is not a product, but a continuous process
of trust, vigilance, and responsibility.

Keep Learning!